

Project 04 — FTP Server (ISP Media Content)

Course: Web Technologies | **Format:** Project | **Duration:** 1 week

Application Overviewd

An FTP server for an Internet Service Provider (ISP) that hosts various media contents (Movies, Software, TV series, Games, etc.) for customers. Three types of users: **ADMIN**, **MODERATOR**, **MEMBER** (general service users).

- **ADMIN:** main control point – adds/deletes moderators, uploads/deletes contents.
- **MODERATOR:** adds/deletes media contents, views all uploaded contents, views content requests.
- **MEMBER** (unregistered service users): browses and downloads contents, searches, filters by category/sub-category, makes content requests via a request box.
- **Registration** is only for ADMIN and MODERATOR. Members do not register; they get read-only access to browse and download.

Team Structure & Student IDs

Student ID	Task	Main Features
	Task 1	User Authentication (Admin/Moderator), Profile Page, Home Page, Category Navigation (view only)
	Task 2	Admin – Add/Delete Moderators, Upload/Delete Contents (full CRUD on users and contents)
	Task 3	Moderator – Add/Delete Media Contents (CRUD on contents), View All Contents, View Requests
	Task 4	Member (unregistered) – Browse by Category/Sub-category, Search, Request Box (create request), Admin/Moderator View Requests

Shared Database Schema (same for all)

Table	Key Columns
users	id, name, email, password_hash, role (admin/moderator), profile_picture, created_at
categories	id, name, parent_id (for sub-categories: e.g., Movies → English, Action, etc.), created_at
contents	id, title, description, file_path, category_id, uploader_id (moderator or admin), download_count, uploaded_at
content_requests	id, requester_ip (or session_id), content_title, category_requested, message, status (pending/fulfilled/rejected), created_at

No code sharing except the schema.

Global Technical Requirements

- PHP MVC (controllers/, models/, views/, config/)
- Passwords hashed with `password_hash()` / verified with `password_verify()`
- PDO/mysqli + **prepared statements** for every query – no string-concatenated SQL
- Server-side validation on every form before any DB write; show inline error messages
- `session_start()` on every page that requires auth; redirect unauthenticated users
- AJAX endpoints return `Content-Type: application/json`; handle errors client-side
- File uploads go to `public/uploads/contents/` – validate MIME type, size, and file extension (e.g., .mp4, .pdf, .exe) server-side
- Do not drop or alter the shared schema tables

Mandatory Grading Criteria (each student must satisfy all 10 within their task)

1. **Basic Web Security** – SQL injection prevention, XSS protection, CSRF awareness, secure password storage.
2. **UI (HTML/CSS)** – Clean, responsive, user-friendly interface.
3. **Feature Completeness** – All assigned requirements work error-free.

4. **DB** – Correct use of shared schema, proper relationships, data integrity.
5. **Auth (Session/Cookie)** – Session management, role-based access, “Remember Me” (optional).
6. **MVC** – Clear separation of business logic, presentation, and request handling.
7. **JS Validation** – Client-side validation on forms (e.g., registration, content upload, request box).
8. **PHP Validation** – Server-side validation on every input before DB write.
9. **Ajax/JSON** – At least one AJAX endpoint returning JSON per student.
10. **Git Contribution** – Feature branches, meaningful commits (≥ 3 per student), merge into `main` via PR.

Git Flow

- Repository has a `main` branch (protected, no direct pushes).
- Each student creates a **feature branch** from `main` named `feature/taskX-studentID` (e.g., `feature/task1-2350434-1`).
- Work on the feature branch, commit regularly (at least 3 commits per student).
- After completing the feature, open a **pull request** into `main` and merge.
- Final `main` must contain all four feature branches merged with full history.

Task Breakdown

Task 1 – User Authentication (Admin/Moderator), Profile Page, Home Page, Category Navigation

Student:

CRUD included: Create (registration for admin/moderator), Read/Update (profile).

Requirements:

- **Registration** – only for ADMIN and MODERATOR. Form collects name, email, password (≥ 8 chars), role (admin/moderator). Validate unique email, hash password. Redirect to login with flash message.
- **Login & Remember Me** – login creates `$_SESSION['user_id']`, `$_SESSION['name']`, `$_SESSION['role']`. “Remember Me” cookie + token for auto-login.
- **Profile Page** (session-gated) – admin and moderator can update name, email, profile picture (upload to `public/uploads/`), change password (require current).
- **Logout & Navbar** – destroy session and cookie. Navbar shows different links based on role (admin, moderator, or guest). For guests (members), no login/register shown – they directly browse.
- **Home Page (general)** – for all visitors (members, admins, moderators):

- Category tabs (Movies, Software, TV series, Games, etc.) – fetched from DB (top-level categories).
 - Highlighted contents – show some most downloaded or recently added contents (name, category, brief description).
- **Category Navigation** – clicking a category shows all contents under that category and its sub-categories. Sub-categories displayed as filter options. Content listing shows title, description, download link (file path).
- **JS validation** on registration and profile forms (password match, email format).

Key outputs: users table with admin and moderator accounts; session management; home page with categories and highlighted contents.

Task 2 – Admin: Add/Delete Moderators, Upload/Delete Contents (full CRUD)

Student:

CRUD included: Full CRUD on users (moderators) and contents.

Requirements:

- **Admin Gate** – every admin page checks `$_SESSION['role'] === 'admin'`; redirect others.
- **Manage Moderators** – list all moderators (role='moderator'). Admin can:
 - Add new moderator (registration form) – creates user with role='moderator'.
 - Delete existing moderator – cascade? Remove their uploaded contents (or reassign to admin).
- **Manage Contents** – admin can upload, edit, delete media contents.
 - Upload form: title, description, category (dropdown with sub-categories), file upload (store in `public/uploads/contents/`). Validate file type/size.
 - Edit – pre-fill form, update record.
 - Delete – remove file from server and DB record.
- **View All Contents** – admin can see all uploaded contents (with uploader info).
- **Admin Dashboard** – counts of total contents, categories, moderators, pending content requests (from Task 4).
- **PHP & JS validation** on upload/edit forms.
- **AJAX** – optional for delete confirmation or dynamic category loading.

Key outputs: Admin can manage moderators and contents; all CRUD operations on users and contents.

Task 3 – Moderator: Add/Delete Media Contents, View All Contents, View Requests

Student:

CRUD included: Create and Delete on `contents` (moderator's own uploads). Read for viewing contents and requests.

Requirements:

- **Moderator Gate** – only users with `role='moderator'` can access moderator pages.
- **Add New Content** – form similar to admin upload: title, description, category, file upload. Saves `uploader_id` as the logged moderator.
- **Delete Content** – moderator can delete **only contents they uploaded** (or admin can allow deletion of any – requirement says "moderator can add or delete media contents" – assume own or all? Safer: moderator can delete any content to simplify, but usually own. We'll specify: moderator can delete any content (admin-like) or own. Use "moderator can add or delete media contents" – no restriction stated, so allow deletion of any.) To be consistent with real FTP servers, we allow moderator to delete any content (but admin overrides).
- **View All Uploaded Contents** – moderator sees full list with search/filter by category.
- **View Content Requests** – moderator can see the request box entries (from Task 4) – list pending requests, mark as "fulfilled" or "rejected" (update `status`). This gives moderator ability to respond to member requests.
- **JS validation** on upload form.
- **AJAX** – for delete confirmation, request status update without page reload.

Key outputs: Moderator can manage contents (add/delete) and view/update content requests.

Task 4 – Member (unregistered): Browse, Search, Filter, Request Box, Admin/Moderator View Requests

Student:

CRUD included: Create (content request by member), Read (search/filter/browse).

Requirements:

- **Member Access** – no login required. Any visitor is a "MEMBER" (unregistered).
- **Browse by Category/Sub-category** – from home page category tabs or sidebar, clicking a category shows all contents under it. Sub-categories (e.g., Movies → Action, Comedy) allow filtering.

- **Search Box** – on home page, search by title or description. AJAX GET `/api/contents/search?q=` returns JSON, dynamically updates content list.
- **Filtering** – alongside search, provide filters: category (dropdown), sub-category (dependent), maybe file type.
- **Content Download** – each content item shows a download link (direct to file or script). Increment download count (optional).
- **Request Box** – on home page, a form for members to request new content:
 - Fields: content title, category (dropdown), additional message.
 - Submit (AJAX POST to `/api/requests/add`) – saves to `content_requests` with `status='pending'` and requester IP or session ID.
 - Show success message.
- **Admin/Moderator View Requests** – for Task 3 (moderator) and Task 2 (admin) integration: they can see the same `content_requests` table and update status. Ensure the page is accessible to admin and moderator roles (via respective dashboards).
- **JS validation** on search filters (valid inputs) and request box (non-empty title).
- **PHP validation** – prevent empty requests, sanitize input.

Key outputs: Unregistered members can browse, search, filter, download, and submit content requests. Admin and moderator can view and manage those requests (status updates).